PROJECT CHEAT SHEET

AI-Powered Resume Screening and Ranking System (Nexus ATS)

Major Project Phase II | Dept. of ISE, VKIT | Mithila R, Sneha K, Yashaswini R | Guide: Prof. Chandrakala V

How to use this sheet: read section 1 and 2 first (the pitch and the big picture), skim 3 to 9 (how each part works), memorise the numbers in 11, and use section 12 (questions and answers) the night before. Section 13 lists what **not** to claim. Every number here comes from our own tests and report.

1. The 30-second pitch

Hiring teams get hundreds of resumes per opening. Normal applicant tracking systems (ATS) only count keywords, so they reward keyword stuffing, cannot tell that FastAPI and Flask are related, give no reason for a rejection, and can carry hidden bias. **Our system ranks candidates on seven weighted signals, treats every skill as a claim that needs evidence, hides identity before scoring (blind screening), and gives every decision an explanation** through a council of four review agents. It is a deployed web application: React on Vercel, FastAPI on Render, MongoDB Atlas for data.

Phase 1 versus Phase 2 (they will ask)

	Phase 1	Phase 2 (this project)
Scoring	Keyword match only	Seven-factor hybrid score with a skill ontology and evidence weighting
Explanation	None	Score breakdown, skill gap, red flags, a four-agent debate and a verdict
Fairness	None	Blind screening, Fair-Hiring Auditor agent, name-swap test (no score change)
Delivery	Single app	Web app + REST API + database, deployed live with sign-in (including Google)

2. The big picture

Technology stack in one table

Layer	What we used	Why
Web interface	React 19, Vite, Tailwind CSS, lucide icons	Fast single-page app; Vite builds it to static files
Hosting: frontend	Vercel	Free static hosting, redeploys on every push to GitHub
API	FastAPI (Python 3.11) served by Uvicorn	Fast, automatic validation, easy REST + docs
Hosting: API	Render (free web service, defined in render.yaml)	Runs Python; redeploys on every push
Database	MongoDB Atlas (free M0 cluster), PyMongo	Accounts, sessions and candidate pools as documents
Legacy data	SQLite + SQLAlchemy + Alembic	Screening history and job tables of the REST service
Core engine	Python: scikit-learn, NumPy, pandas, PyMuPDF, python-docx, pytesseract	Parsing, scoring, council; works offline
Optional AI	Sentence-BERT (all-MiniLM-L6-v2), LLM narrative	Used only when installed; the system works without them
Desktop UI	Streamlit app (app.py)	The original Python interface, runs locally
Tests	pytest + mongomock (Python), Node test runner (web)	104 automated tests

How a request flows

- Recruiter opens the Vercel site, **signs in** (email or Google). The browser calls the Render API; the API checks the account in MongoDB and returns a session token.

- The site loads a **candidate pool** (12, 50 or 100 resumes) with GET /candidates?pool=... from MongoDB, or the recruiter **uploads resumes**, which are parsed in the browser.
- The pages (leaderboard, dossier, comparison, interview assistant, analytics, chatbot) are all computed in the browser from the loaded candidates.
- The heavy scoring (parser, ranker, council) is the **Python core**. Its results for the demo pools were computed offline and stored as seed data in MongoDB; the same core powers the Streamlit app and the REST service.

Two parsers exist, say so confidently: (1) the **Python parser** (src/core/resume_parser.py) is the full, accurate engine used by Streamlit/API and for the test suite; (2) the **in-browser parser** (frontend/src/utils/resumeParserClient.js) is a lighter version so the deployed site can accept uploads without a server round-trip. Both were improved this week (name, experience, evidence, skills).

3. How the scoring works (the heart of the project)

Pipeline, step by step

- **Parse**: PDF, DOCX, TXT (OCR with Tesseract for scanned files) into name, contact, skills, education, experience, projects, certifications.
- **Blind screening** (optional): name becomes 'Candidate #001'; contact details, location, links and gendered words are removed. Only the anonymised copy is scored.
- **Skill ontology**: every skill is normalised (79 canonical skills, 353 aliases, 8 categories) so 'sklearn' = Scikit-Learn, 'k8s' = Kubernetes.
- **Skill gap analysis**: each required skill is matched, partially matched (a related skill, half credit) or missing.
- **Hybrid ranker**: seven scores, weighted, give the match percentage.
- **Council review**: four agents score, cross-examine each other and give a verdict with red flags, confidence and dissent.
- **Explainable output**: leaderboard, score breakdown, skill gap, interview questions, saved history.

The seven weights (memorise)

Signal	Weight	What it measures
Technical skills	35%	Required skills matched through the ontology (partial credit for related skills, evidence-weighted)
Semantic match to the job description	20%	Cosine similarity of embeddings (Sentence-BERT) or TF-IDF when the model is not installed
Experience	15%	Years against the minimum the job needs
Education	10%	Degree and branch against the preferred degrees
Projects	10%	Quality and number of described projects
Certifications	5%	Number of listed certifications (not verified with the issuer)
Resume quality (ATS)	5%	Format and content checks: sections, length, metrics, parse warnings

Weights are configurable. They are chosen by us, **not learned** from recruiter data; the sensitivity test shows rankings stay stable when each weight changes by up to 20%.

Formulas you may be asked to write

Name	Formula	In plain words
Match %	Match = 0.35·S_tech + 0.20·S_sem + 0.15·S_exp + 0.10·S_edu + 0.10·S_proj + 0.05·S_cert + 0.05·S_ats	Weighted sum of seven 0-100 scores
Partial credit	matched = 1, related skill = 0.5, missing = 0	FastAPI asked, Flask found: half credit
Evidence-weighted coverage	(skills shown in experience or projects + 0.5 · skills only listed) / required skills	A claimed skill with no proof counts half
TF-IDF	tf-idf(t,d) = tf(t,d) · idf(t); idf is smoothed: ln((1+N)/(1+df)) + 1	Rare words weigh more than common words

Cosine similarity	cos(a,b) = (a·b) / (|a|·|b|)	Angle between two vectors; 1 = same direction
BM25	sum over query terms: idf(t) · tf·(k1+1) / (tf + k1·(1 - b + b·|d|/avgdl)), k1 = 1.5, b = 0.75	TF-IDF with saturation and length normalisation
Talent search fusion	score = 0.55·dense + 0.45·BM25 (normalised)	Meaning (dense) plus exact words (BM25)
Experience (years)	union of month ranges, so overlapping jobs are not double counted	Jan 2024-Apr 2024 = 4 months
Experience score	if years >= required: 85 + min(15, 5 · surplus); else max(25, 80 · years/required)	Meeting the minimum gives 85+
Kendall's tau	tau = (concordant - discordant pairs) / (n(n-1)/2)	Agreement of two rankings, -1 to +1
Confusion matrix	entry (i,j) = items of reference class i that got verdict j	Used to compare labels with verdicts

Why TF-IDF as fallback? The free server has 512 MB of memory, so the large embedding model is optional. TF-IDF cosine needs no download and works offline; the interface always shows which backend produced the semantic score.

4. The four-agent council

Agent	Weight in consensus	What it judges
Tech Lead	35%	Technical depth: 0.40 evidence-weighted skill coverage + 0.20 depth + 0.10 craftsmanship + 0.10 impact + 0.10 project maturity (blended with GitHub authenticity) + 0.10 breadth
Fair-Hiring Auditor (merit)	25%	Merit only; checks fairness integrity and raises bias flags; never sees the name in blind mode
Culture and Retention	20%	Career timeline: starts at 68 points; adjusts for tenure and progression; gaps are reported but never penalised
Seniority and Compensation	20%	Level L3 to L6 from experience (below 2 yrs L3, below 5 L4, below 9 L5, else L6) and fit with the role level; gives an INR salary band

Consensus, verdict and the gates

- **Consensus = 0.35·TechLead + 0.25·Merit + 0.20·Culture + 0.20·Seniority.** Verdict: **Strong Hire at 80 or more, Hire at 68, Lean Hire at 55, otherwise Reject.** (In the app, Hire and Lean Hire are shown together as Lean Hire.)
- **Knock-out gates** stop a high average from hiding a fatal flaw: evidence-weighted coverage below 25% forces Reject; below 50% caps at Lean Hire; one high-severity red flag, inverted dates or missing education cap at Lean Hire; two high-severity flags force Reject.
- **Confidence** starts at 100 and drops by 1.2 times the spread of the four scores and by penalties (missing timeline 8, no skills 15, inverted dates 15, missing education 10, concurrent jobs 20, under 150 words 10).
- **Dissent**: an agent more than 20 points away from the consensus is reported. **Human review** is requested for high-severity flags or low confidence. The system **supports** the recruiter; it never decides alone.

Red flags you should be able to name

Experience deficit, required skills missing, unverified skill claims, skill stuffing, no quantified impact, date inconsistency (inverted dates), concurrent employment conflict, corrupt format, missing education.

5. Fairness and explainability

- **Blind screening** removes direct identifiers before any scoring. Scores and verdicts did not change in **96 name-substitution runs** (plain and blind mode): maximum change 0.0.
- **Honest limit:** it removes direct identifiers only. Proxies such as the name of a college are not removed, and group-fairness metrics were not measured (no demographic data).

- **Explainability**: every verdict has a score breakdown, matched/partial/missing skills with the reason, red flags with explanations, a replayable debate and a confidence value.
- **Why it matters**: the EU AI Act treats recruitment AI as high-risk (needs transparency and human oversight); Amazon's scrapped recruiting tool is the classic example of hidden bias.

6. The web application: what each page does

Page	Purpose	Good to know
Job Management	Choose the role (8 built-in, or create your own), load a pool or upload resumes	Salary shown in INR only
Screening Leaderboard	Ranked cards with score, evidence line, skills, flags, verdict; filter by verdict; blind switch	Blind mode hides names and photos
Candidate DeepDive	One candidate: four council scorecards, debate replay, GitHub check, compensation band	Shows the reason behind the verdict
Candidate Comparison	Head-to-head matrix of two candidates	Useful when two are close
Interview Assistant	19-24 tailored questions per candidate with 'what to look for'; filters; copy all	**Only for Strong Hire and Lean Hire candidates; rejected ones are not listed**
Recruiter Analytics	Cohort view: verdict donut, spider web, score histogram, experience scatter, skill coverage, red flags	All numbers computed from the loaded pool; click a verdict to focus
Recruiter Chatbot	Ask in plain English: shortlist, compare, why, red flags, salary, draft emails, interview questions	Rule-based and grounded in the loaded pool; remembers context; no LLM needed

The Recruiter Analytics page (new, they may ask what it is for)

Purpose: a bird's-eye view of the whole pool so the recruiter decides where to spend interview time. A **cohort** is a group of candidates with the same verdict. The **spider web** compares the average of six scores (skill match, resume quality, council score, code authenticity, culture fit, seniority fit) for each group, so you see exactly where rejected candidates fall short. The histogram and scatter show how scores and experience are distributed; the bars show which required skills the pool lacks and which red flags repeat.

The chatbot and the interview kit (no LLM)

- **Chatbot:** word-boundary intent matching, history-aware context (pronouns like 'her', 'why?', 'show more'), pagination, asks which person when two share a name, three email types, clickable suggestions. Answers only from the loaded pool, so it cannot invent candidates.
- **Interview kit:** built from the role family, the matched skills, the red flags and the GitHub risk; 19 to 24 questions per candidate (mean 20.7 over 162). Each question has a rubric.

7. Backend, security and database

API endpoints

Endpoint	What it does
GET /health	Health check used by Render; reports database connected
POST /auth/signup	Create an account (Gmail only, password rules)
POST /auth/signin	Email + password; returns a session token
POST /auth/google	Send Google's ID token; the server verifies it, then creates or links the account
GET /auth/me, POST /auth/signout	Who am I / end the session
GET /candidates?pool=core12|benchmark50|benchmark100	Returns a candidate pool from MongoDB
/api/v1/jobs, /resumes, /screenings, /audits	Original REST resources of the screening service (SQLite)

Security design (a favourite review topic)

- **Passwords:** PBKDF2-HMAC-SHA256, 210,000 iterations, random 16-byte salt per user; compared in constant time; never stored in plain text.
- **Sessions:** a random 32-byte token is sent to the browser; **only its SHA-256 hash is stored**; sessions expire after 7 days (MongoDB TTL index).
- **Brute force:** 5 wrong passwords lock the account for 15 minutes (HTTP 429). Unknown email and wrong password give the **same generic message** so attackers cannot find which emails exist.
- **Google sign-in:** the server checks Google's ID token: audience equals our client ID, correct issuer, not expired, email verified, Gmail only. Google-only users have no password hash.
- **Secrets** (database URI, keys) live in environment variables on Render/Vercel, **never in GitHub**. The database password and OAuth secret were rotated after the setup.
- **CORS:** the API accepts browser calls only from the allowed frontend origin (CORS_ORIGINS).

MongoDB (why and what)

Collection	Holds	Important index
users	Name, email, role, company, password hash or Google id	Unique email
sessions	Hashed session tokens	Unique token hash + TTL (auto-delete after expiry)
candidates	162 candidate documents in 3 pools (12 + 50 + 100)	Unique (pool, id); order field keeps the ranking

Why MongoDB: candidate profiles are nested documents (skills lists, council scorecards, debate), which fit documents naturally; Atlas gives a free managed cluster. The pools are seeded automatically (idempotent) from api/seed/*.json the first time the API starts with an empty pool.

8. Deployment, GitHub and 'CI/CD' (answer this carefully)

Where everything lives (all in Yashaswini's accounts)

Part	Service	How it is configured
Source code	GitHub repository (public)	Branch main; every push triggers the two hosts below
Website	Vercel	Build command 'npm run build', output 'dist'; env vars VITE_API_BASE_URL, VITE_AUTH_PROVIDER=api, VITE_CANDIDATES_PROVIDER=api, VITE_GOOGLE_CLIENT_ID
API	Render web service (blueprint render.yaml)	Build 'pip install -r requirements-api.txt'; start 'uvicorn api.main:app'; health check /health; env vars MONGODB_URI, MONGODB_DB, GOOGLE_CLIENT_ID, CORS_ORIGINS
Database	MongoDB Atlas (free M0)	Database user + network access list; connection string stored only in Render
Google sign-in	Google Cloud OAuth client (Web)	Authorised JavaScript origin = the Vercel site; the client ID is public, the client secret is not used by the website

What is our CI/CD? Be exact.

Honest answer: we do **not** have a custom GitHub Actions pipeline. We use **continuous deployment through Git integration**: when we push to the main branch, **Vercel** automatically builds and publishes the website, and **Render** automatically builds and restarts the API (you can see these deployments on GitHub under 'Deployments'). That is the CD part. Tests (pytest and the Node test runner) are run by us before pushing, so CI is manual. **Next step** would be a GitHub Actions workflow that runs both test suites on every push and blocks a bad deploy.

Docker

The repository has a docker-compose.yml for running the stack locally, but **the live deployment does not use Docker**: Render runs the Python code directly and Vercel serves static files. (Containerising would make the environment identical everywhere; it is listed as future work.)

Why the first load can be slow

Render's free plan **puts the API to sleep** after inactivity; waking it takes up to about a minute. The website now pings the API as soon as it opens and shows 'Signing you in… the server may need up to a minute'. Before a demo, open the /health URL once. A paid plan or an uptime pinger removes this.

How to redeploy or roll back

- Normal: commit and push to main; both hosts rebuild by themselves (Vercel about 1 minute, Render about 3-10 minutes).
- Problem on Render: Dashboard, Manual Deploy, 'Clear build cache and deploy'. Problem on Vercel: Deployments tab, promote an older deployment.
- Git rule we used: commits are authored with the account that owns the Vercel project, otherwise Vercel Hobby refuses to deploy.

9. Testing

Suite	Tests	Covers
Python core and API (pytest)	31	Parser, ontology, ranker, council, GitHub inspector, search, database, sign-in flow, UI boot, the four demo resumes
Accounts and candidate API (pytest + mongomock)	18	Sign-up rules, lockout, sessions, Google token checks (simulated), candidate pools
Parser regression tests (pytest)	5	Found with a real resume: no false AWS/Terraform skills, correct projects, certifications, short-job months
Web logic (Node test runner)	50	Interview kit and chatbot on all 162 candidates, in-browser parser, INR-only rule, analytics maths
Total	**104**	All passing

The demo test resumes (live pipeline, no mock data)

Resume	Scenario	Result	Pass?
Gold Standard	Strong ML engineer with evidence	Rank 1, 88.1%, STRONG HIRE, no flags	PASS
Buzzword Exaggeration	35 skills, 4 months experience	41.6%, REJECT, experience deficit + unverified claims + stuffing	PASS
Lateral Partial Match	Flask developer, FastAPI via Flask	61.3%, LEAN HIRE, FastAPI partial, PyTorch missing	PASS
Corrupt ATS Format	Damaged layout, inconsistent dates	ATS 23, 34.0%, REJECT, date flag	PASS

Four more cases (second strong candidate, junior candidate, missing sections, name-swap fairness) also pass: 8 of 8.

10. Everything we changed in the last stretch (explain as 'improvements after testing')

- **Google sign-in** looked stuck after verification because the free server was asleep (73 seconds measured). Fixed with an early wake-up ping and a 'Signing you in…' message.
- **Interview Assistant** listed rejected candidates. Now only Strong Hire and Lean Hire are shown; the chatbot also declines to prepare questions for a rejected candidate.

- **Resume uploader** showed a wrong name, assumed 3 years of experience by default, lost line breaks in PDFs and matched skills inside words. All fixed in the in-browser parser.
- **Python parser** reported skills the resume did not have (ESP32-S3 read as AWS S3, employer 'IAC' read as IaC), split projects wrongly and missed certifications. All fixed with regression tests.
- **Money** is now shown in **INR only** (annual, in lakhs per annum, LPA) everywhere, as advised.
- **Recruiter Analytics** was a static page with fixed numbers; it is now data-driven with interactive charts (donut, spider web, histogram, scatter, bars).
- **Account security and storage** moved from browser-only to a real server with MongoDB; Google tokens are verified on the server.

11. Numbers to remember

Fact	Value
Time to screen one resume (offline)	about 30 ms (parse 18, rank 3.8, council 7.8); roughly 34 resumes per second on one CPU thread
Keyword baseline vs our ranking (12-pool)	Kendall tau 0.667: they agree on the extremes and differ where the design intends
Weight sensitivity	200 random weight changes of up to 20%: top candidate stayed first every time; mean Kendall tau 0.969
Name invariance	96 name-substitution runs, maximum score change 0.0 (plain and blind)
Evidence weighting ablation	Removing it turns two REJECT verdicts into LEAN HIRE (the buzzword resume and one DevOps resume)
12-resume core pool	2 Strong Hire, 3 Lean Hire, 7 Reject; 5 shortlisted (41.7%); top match 88.1%
50-resume benchmark (Senior ML Engineer)	Top match 93.2%, average 56.9%; 10 Strong Hire, 10 Lean Hire, 30 Reject; 30 resumes carry at least one flag
Total candidates stored in MongoDB	162 (12 + 50 + 100) in three pools
Skill ontology	79 skills, 353 aliases, 8 categories, 15 related-skill pairs
Interview questions per candidate	19 to 24 (mean 20.7)
Verdict thresholds	80 Strong Hire, 68 Hire, 55 Lean Hire, below 55 Reject
Password hashing	PBKDF2-HMAC-SHA256, 210,000 iterations; lockout 5 tries / 15 min; session 7 days
Automated tests	104 (54 Python + 50 web), all passing

If the reviewer compares the app's 100-resume pool with the report: the pools in the app are **precomputed demo data** stored in the database; the evaluation numbers in the report (Chapter 8) come from running the live pipeline. Quote the report for evaluation results.

12. Likely questions and short answers

About the idea and the data

Q. What problem does your project solve?

Keyword ATS tools rank by word overlap, cannot judge whether a skill claim is real, give no explanation and can be biased. We give an explainable, evidence-based, bias-aware ranking with a reason for every decision.

Q. How is it different from Phase 1?

Phase 1 only matched keywords. Phase 2 adds the skill ontology, seven-factor hybrid score, evidence weighting, blind screening, a four-agent council with explanations, a deployed web app with real sign-in and a database.

Q. Is this machine learning? Where is the training?

It is a hybrid of rule-based scoring and optional pretrained language models. We do not train a model, because we have no labelled real hiring data. That keeps every decision explainable and deterministic; we say so as a limitation and list learned weights as future work.

Q. What data did you use? Is it real?

Synthetic resumes (12, 50 and 100) written for specific scenarios (strong, buzzword-stuffed, corrupt, lateral move, etc.). We state clearly that results show consistency and mechanism, not real-world hiring accuracy.

Q. What is your accuracy?

We do not claim an accuracy number, because there is no ground truth from real recruiters. We report behaviour tests: the strong resume ranks first, stuffed and corrupt resumes are rejected, scores are stable under weight changes and do not change with names.

Q. Why synthetic data?

Real resumes contain personal data and cannot be shared or published; synthetic ones let us test specific cases and make the experiments reproducible.

About scoring and NLP

Q. What is TF-IDF and why use it?

Term frequency times inverse document frequency: words that are frequent in a resume but rare across documents get high weight. It is fast, offline and a good fallback when the sentence-embedding model is not installed.

Q. What is the difference between TF-IDF, BM25 and embeddings?

TF-IDF and BM25 match exact words (BM25 saturates term frequency and normalises for length). Embeddings (Sentence-BERT) capture meaning, so 'built REST services' matches 'developed HTTP APIs'. Our talent search fuses both: 0.55 dense + 0.45 BM25.

Q. What is cosine similarity?

The cosine of the angle between two vectors; 1 means same direction (very similar), 0 means unrelated.

Q. What is the skill ontology?

A table of 79 canonical skills with 353 aliases and 15 related pairs. It normalises names and gives half credit for related skills (FastAPI and Flask).

Q. What does 'evidence-weighted' mean?

A skill that appears in experience or project text earns full credit; a skill that appears only in the skills list earns half. This is what stops keyword stuffing.

Q. Why these weights (35, 20, 15, 10, 10, 5, 5)?

Skills are the strongest signal, semantic fit second, then experience; education, projects, certifications and quality are supporting signals. They are heuristics chosen by us and configurable; the sensitivity test shows rankings are stable within plus or minus 20%.

Q. What is the council and why four agents?

Four specialised reviewers (Tech Lead, Fair-Hiring Auditor, Culture and Retention, Seniority and Compensation) score independently, cross-examine each other and a weighted consensus gives the verdict. Separate roles keep the reasons separate and show disagreement (dissent) instead of hiding it.

Q. Does the council use an LLM?

No. It is deterministic and rule-based. An LLM can optionally polish the wording, but it can move a score by at most 10 points, and the system works fully without it.

Q. What are knock-out gates?

Rules that stop a high average from hiding a fatal flaw, for example evidence coverage below 25% forces Reject, inverted dates cap the verdict at Lean Hire.

Q. How do you handle scanned resumes?

OCR with Tesseract when no text can be extracted from the PDF; the result is marked as OCR-based.

About fairness

Q. How do you reduce bias?

Blind screening removes names, contact details, location, links and gendered words before scoring; a Fair-Hiring Auditor agent checks the decision; and the name-swap test proves scores do not change with the name (96 runs, change 0.0).

Q. Is your system completely fair?

No. It removes direct identifiers only; proxies like college names remain and we could not measure group fairness without demographic data. We list this as a limitation and require human review.

Q. Will it replace recruiters?

No. It is decision support: it ranks, explains and flags, and asks for human review when confidence is low.

About the application

Q. Why React and FastAPI?

React gives a fast interactive interface and Vercel hosts it for free. FastAPI is quick, validates input automatically and fits our Python scoring core.

Q. Why two parsers (browser and Python)?

The Python parser is the accurate engine for the API and tests. The browser parser lets the deployed site accept uploads instantly without sending files to a server, at the cost of being simpler.

Q. Why is the interview assistant empty for rejected candidates?

Interview questions are prepared only for candidates who move forward (Strong Hire or Lean Hire); preparing interviews for rejected candidates would be wasted effort. The page says how many were hidden.

Q. Why INR only?

Our guide advised showing compensation in Indian rupees. All bands are in LPA (lakhs per annum).

Q. How does the chatbot work without an LLM?

It detects the intent with word-boundary rules, finds the candidate from the question or the previous answer, and builds the answer from the loaded data. That is why it cannot make up candidates.

About deployment, GitHub, Vercel, Render, MongoDB

Q. Explain your deployment architecture.

GitHub holds the code. Vercel builds the React site and serves it. Render runs the FastAPI service. MongoDB Atlas stores users, sessions and candidates. The browser talks to Render over HTTPS; Render talks to Atlas with a secret connection string held in an environment variable.

Q. Do you have CI/CD?

We have continuous deployment: every push to main automatically redeploys Vercel and Render. Tests are run manually before pushing. Adding GitHub Actions to run tests on every push is our next step.

Q. Did you use Docker?

There is a docker-compose.yml for local use, but the live deployment runs without Docker on Render's Python runtime. Containerising is planned future work.

Q. Why does the site sometimes take a minute?

Render's free plan sleeps the API when idle and needs up to a minute to wake it. We added an early wake-up ping and a progress message. A paid plan removes this.

Q. How are secrets protected?

They are environment variables on Render and Vercel, not in the repository. The .env.example lists names only. We rotated the database password and OAuth secret after setup.

Q. What is CORS and why did you set it?

A browser rule that blocks a website from calling another domain's API unless the API allows it. We allow only our Vercel site.

Q. Why MongoDB and not SQL?

Candidate profiles are nested documents (skills, scorecards, debate), which map naturally to MongoDB; Atlas gives a free managed cluster. The original REST resources still use SQLite/SQLAlchemy with Alembic migrations.

Q. How does Google sign-in work?

Google shows its own button and returns an ID token to the browser. The browser sends it to our API, which verifies audience, issuer, expiry, verified email and Gmail domain, then creates or links the account and returns our own session token.

Q. How are passwords stored?

PBKDF2-HMAC-SHA256 with 210,000 iterations and a random salt per user; only the hash is stored.

Q. What happens after 5 wrong passwords?

The account is locked for 15 minutes (HTTP 429). A wrong password and an unknown email give the same message.

Q. How would you scale it?

Move off the free tier (always-on instances), add a task queue for large uploads, cache embeddings, containerise with Docker, add GitHub Actions and monitoring, and load-test (not done yet).

About testing and limits

Q. How did you test it?

104 automated tests: Python (parser, ranker, council, API, accounts with an in-memory MongoDB) and web (interview kit, chatbot, parser, analytics). Plus eight scenario resumes run through the real pipeline, and manual checks of the live site.

Q. What did you find when you tested with a real resume?

The uploader mis-read the name, assumed 3 years of experience and the Python parser produced false skills. We fixed these and added regression tests. This shows testing with real input mattered.

Q. What are the limitations?

Synthetic data only; weights are heuristic, not learned; 79-skill ontology built for software roles (embedded or robotics skills are not recognised); blind screening covers direct identifiers only; no usability study; free hosting sleeps; not load-tested.

Q. What is the future scope?

Role-based access and consent controls, validating weights on real labelled data, LinkedIn and portfolio checks, multi-language resumes, email and ATS integrations, GitHub Actions CI, Docker, load testing.

13. What NOT to claim (keeps you safe)

- Do **not** say the model is trained or has a percentage accuracy. Say 'behaviour tests on synthetic data'.
- Do **not** say we have a GitHub Actions pipeline or that we deployed with Docker. Say 'continuous deployment through Git integration'.
- Do **not** say the system is fully unbiased. Say 'it removes direct identifiers and we proved name invariance'.

- Do **not** say it works for every role. The skill ontology is built for software and data roles.
- Do **not** say the free server is always fast. Say 'it sleeps when idle; we wake it before a demo'.
- If you do not know a number, say 'it is in Chapter 8 of the report' and open the report.

14. Abbreviations quick list

Short	Full form	Short	Full form
ATS	Applicant Tracking System	BM25	Best Matching 25 (ranking function)
TF-IDF	Term Frequency - Inverse Document Frequency	SBERT	Sentence-BERT
JD	Job Description	OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation	XAI	Explainable AI
API / REST	Application Programming Interface / Representational State Transfer	CORS	Cross-Origin Resource Sharing
PBKDF2	Password-Based Key Derivation Function 2	TTL	Time To Live (auto-expiry)
LPA	Lakhs Per Annum (INR)	CI/CD	Continuous Integration / Continuous Deployment
JWT / token	Session token (ours is a random token, not a JWT)	RQ	Research Question